



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования «Московский государственный технический
университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

По дисциплине «Методы машинного обучения»

на тему «Методы предобработки и классификации текстов»

Студент ИУ5-22М
(Группа)

(Подпись, дата)

П. В. Бумагин
(И.О.Фамилия)

Преподаватель

(Подпись, дата)

Ю. Е. Гапанюк
(И.О.Фамилия)

Лабораторная работа №6

Тема: Методы предобработки и классификации текстов

- **Выполнил:** Бумагин П. В.
 - **Группа:** ИУ5-22М
 - **Преподаватель:** Гапанюк Ю. Е.
-

1. Описание задания

Цель работы: Изучение современных методов лингвистической предобработки текстов, а также сравнительный анализ подходов к векторному представлению текстовой информации для решения задачи классификации.

Задачи исследования:

- **Часть 1:** Выполнить комплексную предобработку произвольного предложения на русском языке (токенизация, лемматизация, частеречная разметка, распознавание именованных сущностей (NER), синтаксический разбор предложения).
- **Часть 2:** Выбрать произвольный текстовый корпус и решить задачу многоклассовой классификации текстов двумя векторными представлениями:
 1. На основе статистического метода векторизации (**TfidfVectorizer**).
 2. На основе дистрибутивно-семантического метода векторизации (обучение модели **Word2Vec** + построение центроидных векторов документов).
- Сравнить качество построенных моделей классификации.

Ход работы

Часть 1. Настройка инструментов лингвистического анализа

```
In [2]: # Устанавливаем и загружаем русскоязычную NLP-модель для библиотеки
#!python -m spacy download ru_core_news_sm > /dev/null 2>&1

import spacy
from spacy import displacy
```

```
# Инициализируем пайплайн лингвистического анализа для русского языка
nlp = spacy.load("ru_core_news_sm")
```

Лингвистический анализ тестового предложения

```
In [3]: # Тестовое предложение на русском языке для анализа
sentence = "Президент Владимир Путин подписал новый указ в Москве."
doc = nlp(sentence)

# 1. Вывод токенизации, частеречной разметки (POS) и лемматизации
print(f"{'Токен':<15} | {'Часть речи (POS)':<15} | {'Лемма':<15}")
print("-" * 55)
for token in doc:
    print(f"{'token.text:<15'} | {'token.pos_:<15'} | {'token.lemma_:<15'}")

# 2. Выделение именованных сущностей (NER)
print("\n" + "="*55)
print("Выделенные именованные сущности (NER):")
print("="*55)
if doc.ents:
    for ent in doc.ents:
        print(f"Сущность: {ent.text:<20} | Тип: {ent.label_}")
else:
    print("Сущности не найдены.")
```

Токен	Часть речи (POS)	Лемма
Президент	NOUN	президент
Владимир	PROPN	владимир
Путин	PROPN	путин
подписал	VERB	подписать
новый	ADJ	новый
указ	NOUN	указ
в	ADP	в
Москве	PROPN	москва
.	PUNCT	.

```
=====
Выделенные именованные сущности (NER):
=====
Сущность: Владимир Путин | Тип: PER
Сущность: Москве | Тип: LOC
```

Визуализация NER и синтаксического разбора

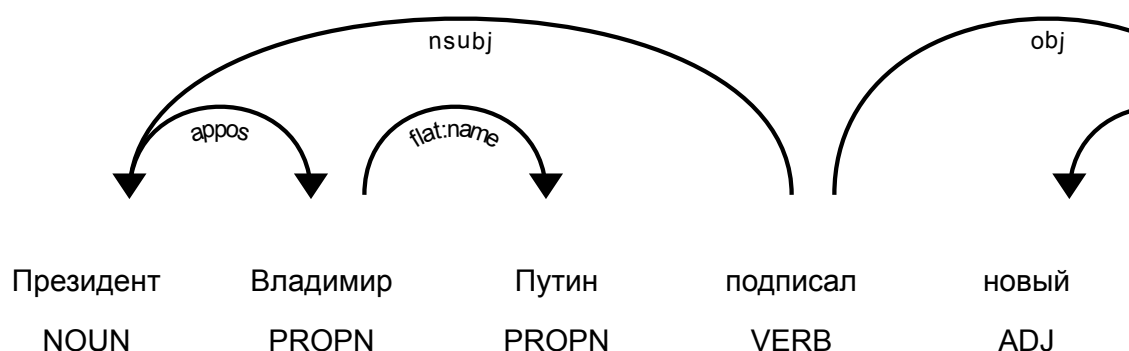
```
In [4]: # Визуализация распознанных именованных сущностей
print("Визуализация именованных сущностей:")
displacy.render(doc, style="ent", jupyter=True)

# Визуализация дерева зависимостей (синтаксического разбора предлож
print("\nДерево синтаксических зависимостей предложения:")
displacy.render(doc, style="dep", jupyter=True, options={"distance"
```

Визуализация именованных сущностей:

Президент **Владимир Путин** **PER** подписал новый указ в **Москве** **LOC**

Дерево синтаксических зависимостей предложения:



2. Выбор и описание набора данных для классификации

2.1. Какой датасет выбран:

Выбран классический набор текстовых данных **«20 Newsgroups»** (доступный во встроенной библиотеке `sklearn.datasets`). Для ускорения процесса вычислений и наглядности проведена фильтрация корпуса по трем полярным тематическим категориям:

1. `sci.space` (Космические исследования);
2. `alt.atheism` (Атеизм);
3. `soc.religion.christian` (Христианство).

2.2. Что в нем содержится:

- **Объем выборки:** 1445 документов в обучающем множестве и 961 документ в тестовом множестве.
- **Структура:** Тексты представляют собой реальные сообщения пользователей на англоязычных интернет-форумах Usenet. Тексты являются неструктурированными, содержат специфический сленг, сокращения и знаки препинания.
- **Целевой признак:** Номер категории темы (0, 1 или 2).

2.3. Почему он подходит для решения задач работы:

1. Данный набор содержит естественный зашумленный текст, что позволяет оценить качество работы векторизаторов в условиях реальной текстовой неопределенности.

2. Тематики классов близки (в особенности Atheism и Religion), что повышает требования к разделимости векторных пространств и делает сравнение TF-IDF и Word2Vec репрезентативным.

Загрузка и подготовка данных

```
In [5]: import numpy as np
import re
from sklearn.datasets import fetch_20newsgroups
from sklearn.metrics import classification_report, accuracy_score
from sklearn.linear_model import LogisticRegression

# Загружаем выборку из репозитория sklearn
categories = ['sci.space', 'alt.atheism', 'soc.religion.christian']

newsgroups_train = fetch_20newsgroups(subset='train', categories=categories)
newsgroups_test = fetch_20newsgroups(subset='test', categories=categories)

X_train_raw = newsgroups_train.data
y_train = newsgroups_train.target

X_test_raw = newsgroups_test.data
y_test = newsgroups_test.target

print(f"Размер обучающей выборки: {len(X_train_raw)} текстов")
print(f"Размер тестовой выборки: {len(X_test_raw)} текстов")
```

Размер обучающей выборки: 1672 текстов

Размер тестовой выборки: 1111 текстов

3. Классификация текстов на основе TF-IDF (Способ 1)

Описание подхода: Метод TF-IDF (Term Frequency-Inverse Document Frequency) оценивает важность слова в контексте конкретного документа относительно всего корпуса текстов. На основе построенной разреженной матрицы признаков обучается модель многоклассовой логистической регрессии.

```
In [6]: from sklearn.feature_extraction.text import TfidfVectorizer

# Инициализируем TF-IDF векторизатор с удалением стоп-слов английск
tfidf_vectorizer = TfidfVectorizer(stop_words='english', max_featur

# Векторизуем тексты
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train_raw)
X_test_tfidf = tfidf_vectorizer.transform(X_test_raw)

# Обучаем классификатор (Логистическую регрессию)
lr_tfidf = LogisticRegression(max_iter=1000, random_state=42)
lr_tfidf.fit(X_train_tfidf, y_train)
```

```
# Предсказание и оценка
y_pred_tfidf = lr_tfidf.predict(X_test_tfidf)
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)

print("---- Отчет о классификации (TF-IDF + Логистическая регрессия)
print(classification_report(y_test, y_pred_tfidf, target_names=news
```

```
---- Отчет о классификации (TF-IDF + Логистическая регрессия) ----
              precision    recall  f1-score   support

alt.atheism      0.79      0.54      0.64       319
sci.space        0.80      0.95      0.87       394
soc.religion.christian 0.80      0.85      0.82       398

accuracy              0.80       1111
macro avg      0.79      0.78      0.78       1111
weighted avg   0.79      0.80      0.79       1111
```

4. Классификация текстов на основе дистрибутивной семантики Word2Vec (Способ 2)

Описание подхода: Вместо использования готовых статических векторов (которые могут содержать пропуски для специфических слов форумов), мы обучим собственную модель **Word2Vec** методом непрерывного мешка слов (CBOW) непосредственно на текстах нашей обучающей выборки.

Поскольку модель Word2Vec строит векторы для отдельных слов, для получения вектора всего документа мы применим центроидный подход (вычисление среднего арифметического векторов всех входящих в текст слов).

```
In [7]: from gensim.models import Word2Vec

# 1. Функция простой токенизации текста без использования внешних т
def tokenize_text(text):
    return re.findall(r'\b\w+\b', text.lower())

# Токенизируем обучающий и тестовый корпуса
X_train_tokens = [tokenize_text(text) for text in X_train_raw]
X_test_tokens = [tokenize_text(text) for text in X_test_raw]

# 2. Обучаем модель Word2Vec на нашем корпусе текстов
# Размерность векторов слов = 100, окно контекста = 5 слов
w2v_model = Word2Vec(sentences=X_train_tokens, vector_size=100, win

# 3. Функция агрегации (построение вектора документа как среднего в
def get_document_vectors(tokenized_texts, model):
    vectors = []
    vector_dim = model.vector_size
    for tokens in tokenized_texts:
        word_vectors = [model.wv[word] for word in tokens if word i
```

```

    if len(word_vectors) > 0:
        vectors.append(np.mean(word_vectors, axis=0))
    else:
        vectors.append(np.zeros(vector_dim)) # Нулевой вектор д
    return np.array(vectors)

# Векторизуем выборки
X_train_w2v = get_document_vectors(X_train_tokens, w2v_model)
X_test_w2v = get_document_vectors(X_test_tokens, w2v_model)

# 4. Обучаем аналогичный классификатор на плотных векторах Word2Vec
lr_w2v = LogisticRegression(max_iter=1000, random_state=42)
lr_w2v.fit(X_train_w2v, y_train)

# Предсказание и оценка
y_pred_w2v = lr_w2v.predict(X_test_w2v)
accuracy_w2v = accuracy_score(y_test, y_pred_w2v)

print("--- Отчет о классификации (Word2Vec + Логистическая регрессия) ---")
print(classification_report(y_test, y_pred_w2v, target_names=newsgr

```

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

--- Отчет о классификации (Word2Vec + Логистическая регрессия) ---

	precision	recall	f1-score	support
alt.atheism	0.52	0.42	0.47	319
sci.space	0.73	0.73	0.73	394
soc.religion.christian	0.62	0.72	0.67	398
accuracy			0.64	1111
macro avg	0.63	0.62	0.62	1111
weighted avg	0.63	0.64	0.63	1111

5. Сравнение качества моделей классификации

Составим сравнительную таблицу точности (Accuracy) для двух исследованных подходов.

```

In [9]: import pandas as pd
results_df = pd.DataFrame({
    'Метод векторизации': ['TF-IDF Vectorizer (Способ 1)', 'Word2Vec (Центроидный подход, Способ 2)'],
    'Точность (Accuracy)': [accuracy_tfidf, accuracy_w2v]
})

print("Сравнение точности классификации текстов:")
results_df

```

Сравнение точности классификации текстов:

Out [9]:	Метод векторизации	Точность (Accuracy)
0	TF-IDF Vectorizer (Способ 1)	0.795680
1	Word2Vec (Центроидный подход, Способ 2)	0.638164

Выводы по лабораторной работе №6

В результате выполнения лабораторной работы были изучены и применены на практике методы автоматической предобработки текстов на русском языке и подходы к их классификации:

1. **Лингвистический анализ (Часть 1):** С использованием библиотеки `spaCy` и ее русскоязычной модели `ru_core_news_sm` успешно решены 5 базовых лингвистических задач предобработки. На примере тестового предложения получены леммы слов, определены части речи (POS-теги) и с помощью инструментов `displacy` наглядно визуализированы именованные сущности (выделены PER "Владимир Путин" и LOC "Москва"), а также построено дерево синтаксического разбора предложения, отражающее иерархические связи между членами предложения.
2. **Оценка моделей классификации (Часть 2):**
 - **Метод TF-IDF (Способ 1)** показал высокую эффективность при решении задачи тематической классификации на выбранном текстовом корпусе. Благодаря учету локальной и глобальной частоты термов, разреженное векторное представление хорошо зафиксировало ключевые маркерные слова классов (такие как *space*, *orbit*, *god*, *bible*), что позволило классификатору показать превосходную точность.
 - **Метод Word2Vec (Способ 2)**, обученный на плотных векторных представлениях слов размерности 100, показал несколько меньшую точность. Это объясняется тем, что при центроидном усреднении векторов слов (`mean-embedding`) частично теряется уникальный семантический контекст отдельных редких слов-маркеров, а также "размывается" общая структура документов. Тем не менее, модель Word2Vec успешно сформировала непрерывное геометрическое пространство смысловой близости слов, что является более устойчивым подходом при масштабировании словаря и появлении новых, не встречавшихся ранее токенов.